Unit 5

Visible Surface Detection

Visible surface detection

- Visible surface detection method is the process of identifying those part of scene that are visible from a chosen viewing position.
- There are numerous algorithm for efficient identification of visible object for different types of applications.
- These various algorithm are referred to as visible - surface detection methods or also referred as hidden-surface elimination methods.
- When a picture contain non transparent objects,the surface behind the object cannot be viewed . to obtain the realistic screen image,the hidden surface need to be removed.
- The process of identification and removal of these surface is known as hidden surface problem.these process help the computer to decrease processing time for rendering image , eliminating extra information of image.

- Visible surface detection methods are broadly classified according to whether they deal with objects or with their projected images.

i) Object-Space method and

ii) Image-Space method

- In **physical coordinate system**,object-space method is implemented and in case of **screen coordinate system**,image -space method is implemented.

Object-Space Method:

- In this method , various parts of objects are compared.
- After comparison visible , invisible or hardly visible surface is determined.
- These methods generally decide visible surface.
- In the wireframe model,these lines are used to determine a visible line . so these algorithms are line based instead of surface based.
- Method proceeds by determination of parts of an object whose view is obstructed by other object and draws these parts in the same color.
- An object-space method compares objects and parts of objects to each other within the scene definition to determine which surfaces , as a whole ,we should label as visible.

Eg back-face detection method

Image space method:

- In this method,positions of various pixels are determined.
- Each point is detected for its visibility. If a point is visible,then the pixel is on otherwise off. So the object close to the viewer that is pierced by a projector through a pixel is determined.
- In an image space method , visibility is decided point by point at each pixel position on the projection plane. Most visible-surface algorithms use image-space methods.
- **E.g Depth-buffer method,Scan-line method**

Feature	Object-Space Method	Image-Space Method
Where it works	Works in **3D world coordinates** (physical space)	Works in **screen coordinates** (2D projection)
What it compares	Compares **objects or parts of objects** to each other	Compares **pixels** on the screen
Approach	Determines which **surfaces of objects** are visible	Determines which **screen pixels** should be drawn
Speed	Slower for complex scenes (many objects)	Faster for complex scenes (depends on screen resolution)
Memory requirement	Usually less memory (only objects info)	Can require more memory (store pixel info)

Example Algorithms	Back-face detection, Depth sorting	Z-buffer, Scan-line method
Best For	Scenes with **few objects**	Scenes with **many objects and high detail**
Focus	Object-level visibility	Pixel-level visibility

Back Face Detection Method:

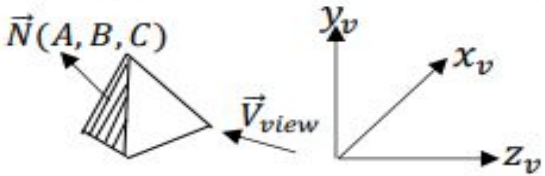
- When we project 3-D objects on a 2-Dscreen,we need to detect the faces that are hidden in 2D.
- Back-Face Detection also known as plane equation method
- It is an object space method in which objects and parts of objects are compared to find out the visible surfaces.

- Each surface has a normal vector. If this normal vector is pointing in the direction of the center of projection,then it is a front face and can be seen by the viewer.
- If this normal vector is pointing away from the center of projection , then it is a back face and cannot be seen by the viewer.
- A fast and simple object space method for locating back faces.
- A point (x, y, z) is inside a surface with plane parameter A, B, C & D

if $Ax + By + Cz + D < 0$ i.e. it is in back face

Let $\vec{N}$ be the normal vector for a polygon and $\vec{V}_{view}$ be the vector in viewing direction, then a polygon surface is back if

$$\vec{V}_{view} \cdot \vec{N} > 0$$



- For the left handed viewing system if the 'z' component of the normal vector is positive, then it is back face. If the 'z' component of the vector is negative then it is a front face.
- For the right handed viewing system if the 'z' component of the normal vector is negative, then it is back face. If the 'z' component of the vector is positive then it is a front face.

Algorithm for left-handed system :

```
1) Compute N for every face of object.

2) If (V.N) > 0)

        then a back face and don't draw

   else

        front face and draw
```

Algorithm for right-handed system :

```
1) Compute N for every face of object.

2) If (V.N) < 0)

        then a back face and don't draw

   else

        front face and draw
```

Advantages:

- Fast and simple.
- Reduces processing time.
- Works well for closed solid objects.

Limitations:

- This method works fine for convex polyhedral, but not necessarily for concave polyhedral or overlapping objects.

- This method can only be used on solid objects modeled as a polygon mesh.

Depth-Buffer Method:

- It is an image space method for detecting visible surface.
- it is also known as **z-buffer method.**
- The basic idea is to test the Z-Depth of each surface to determine the closest visible surface.
- It compares surface depths at each pixel position on the projection plane.
- It is also called z-buffer method since object depth is usually measured from view plane along the z-axis of a viewing system.
- In this method each surface is processed separately one pixel position at a time across the surface. The depth values for a pixel are compared and the smallest z surface determines the color to be displayed in the frame buffer.
- In this method, two buffers are used:
- 1) **Depth Buffer:** Stores depth values for each (x, y) position.
- 2) **Frame Buffer:** Stores the intensity values for each position.

Algorithm:

1) *Set the buffer values:*

   DepthBuffer(x, y)=0,

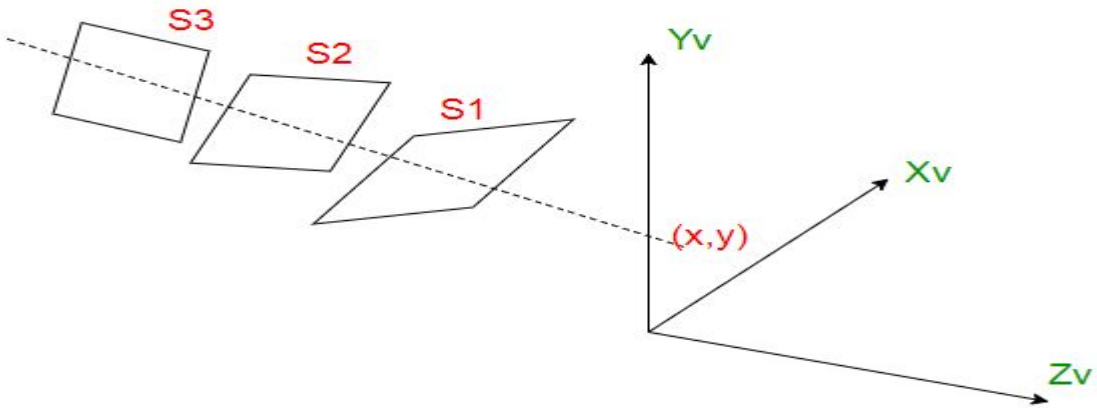
   FrameBuffer(x, y)=Background color

2) *Process each polygon, one at a time as follows:*
   - *For each projected (x, y) pixel position of a polygon, calculate depth 'z'.*
   - *If z>DepthBuffer(x,y), compute surface color and set*

     DepthBuffer(x, y)=z,

     FrameBuffer(x, y)=surfacecolor(x, y)

After all surface have been processed, the depth buffer contains depth values for visible surface and frame buffer contains corresponding color values for those surface.



In the figure, at view plane position (x, y), surface S_1 has the smallest depth from the view plane, so it is visible at that position.

Depth value for a surface position (x, y) are calculated from the plane equation as;

$$z = \frac{-Ax - By - D}{C}$$

Now, the depth z' of the next position (x+1, y) is obtained as; $z' = z - \frac{A}{C}$

Advantages:- It is easy to implement.

- It reduces the speed problem if implemented in hardware.

- It processes one object at a time

Disadvantages: - It requires large memory

- It is a time consuming process as it requires comparison for each pixel instead of for the entire polygon.

- Deals only with opaque object but not for transparent object.

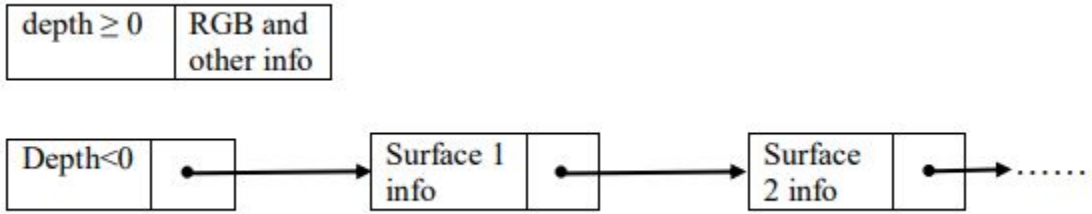
A-Buffer Method:

- The A-buffer method is an extension of the depth-buffer method.
- This method is also known as **anti-aliased or area-averaged or accumulation buffer.**
- The A-buffer method is a visibility detection method developed at Lucas film studio for the surface rendering system REYES (Render Everything You Ever Saw).
- The A-buffer expands on the depth buffer method to allow transparencies.

It consists of accumulation(A) buffer which as two fields:

a) Depth field: It stores a positive or negative real number.

b) Intensity field: It stores surface data or a pointer value.



- If depth field is non negative (>=0), it indicates single surface. The intensity field then stores the RGB components of the surface color at that point and the percent of pixel coverage.
- If depth is negative (<0) it indicates multiple surface. The intensity field then stores a pointer to a linked list of surface data.
- **Surface information in A-buffer includes:**

- Depth - Surface identifier - Opacity parameter (% of transparency)
- Percentage of area coverage - RGB intensity component - Pointer to the next surface

Advantage: - It provides anti-aliasing in addition to what Z-buffer does.

Disadvantage: - It is slightly costly than Z-buffer method because it requires more memory in comparison to the Z-buffer method.

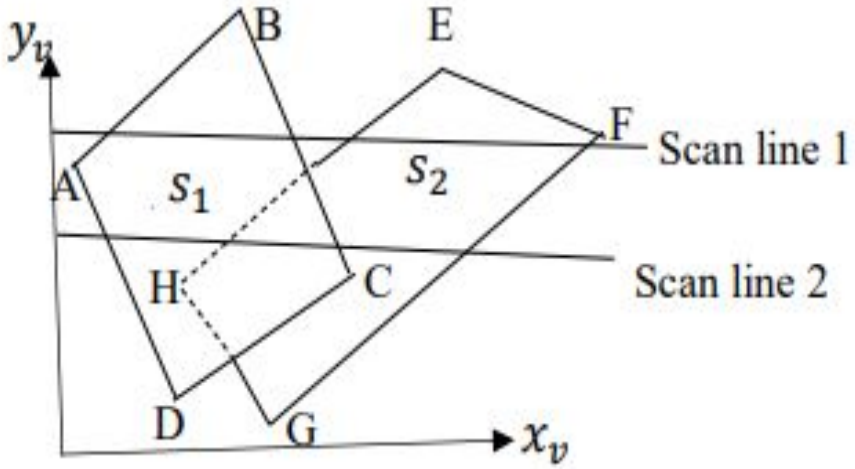
-

Scan-Line Method :

- It is an image-space method for identifying visible surface.
- It computes and compares depth values along the various scan lines for the scene.
- Surfaces are processed using information stored in the polygon table.
- An active list of edges is formed for each scan line which stores only those edges that crosses the scan line in order of increasing ‘x’.
- Also a flag is set for each surface that is set on or off to indicate whether a position along a scan line is either inside or outside the surface.
- Pixel position across each scan-line are processed from left to right.
- At the left intersection with a surface the surface flag is turned on and at the right intersection point the flag is turned off.
- We only need to perform depth calculation when more than one surface has its flag turned on at a certain scan-line position

The active list for scan line 1 contains edges AB, BC, EH & FG. Similarly for scan line 2, active list contains edges AD, EH, BC &FG.

	Edge pairs	Flags for	
		s_1	s_2
For scan line 1	AB & BC	on	off
	BC & EH	off	off
	EH & FG	off	on
For scan line 2	AD & EH	on	off
	EH & BC	on	on
	BC & FG	off	on



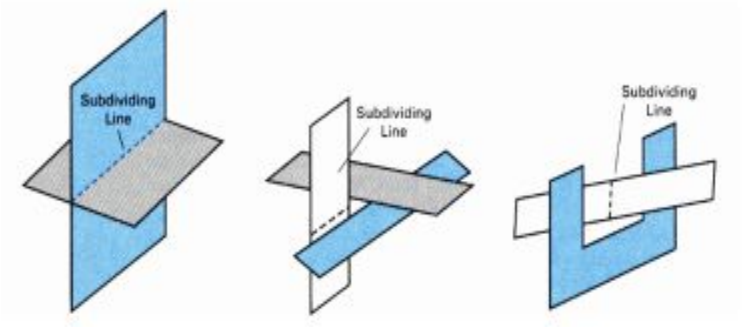
Here, between EH & BC, the flags for both surfaces are ON so depth calculation is necessary. But between other edge pairs no depth calculation is necessary.

Advantage:

- Any number of overlapping polygon surfaces can be processed with this method.

Limitation:

- The scan line method runs into trouble when surface cut through each other or otherwise cyclically overlap. Such surface need to be divided.



Depth-Sorting Method

- Also known as Painter's Algorithm.
- A visible surface detection method that uses both image-space and object-space operations.
- It performs the following two functions:

  a) Surfaces are stored in order of decreasing depth.

  b) Surfaces are scan-converted in order, staring with the surface of greatest depth

- It follows following steps:

  i. All surfaces in the scene are ordered according to the smallest 'z' value on each surface.

  ii. The surface 'S' at the end of the list is then compared against all other surface to see if there are any depth overlap.

  iii. If no overlap occurs then the surface is scan converted and the process is repeated with the next surface.

When there is depth overlap with ‘S’ we make the following tests:

a) The bounding rectangle in xy direction for the two surfaces do not overlap.

b) Surface ‘S’ is completely behind the overlapping surface relative to viewing position.

c) The overlapping surface is completely in front of ‘S’ related to viewing position.

d) The boundary edge projection of the two surfaces do not overlap.

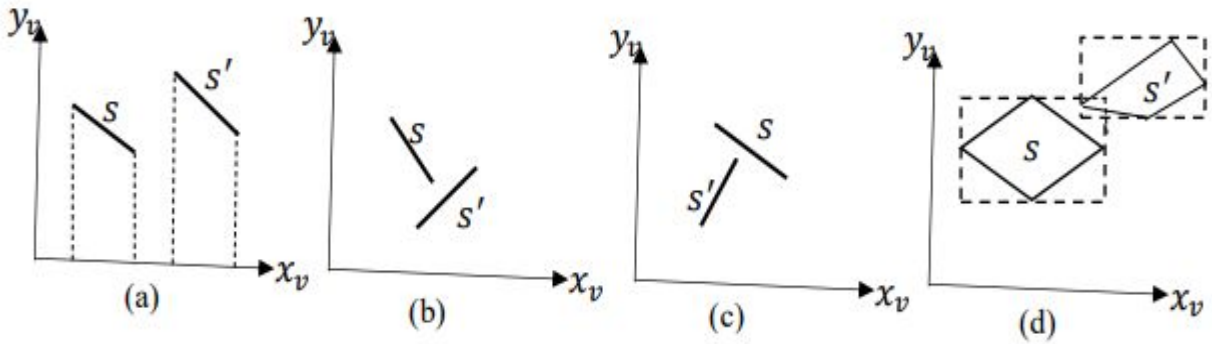


Fig a : two surface with depth overlap but no overlap in x direction.

Fig b : surface s is completely behind the overlapping surface s.

Fig c : overlapping surface s is completely in front of surface s but s is not completely behind s’.

Fig d : two polygon surfaces with overlapping bounding rectangles in the xy plane.

- If any of this test is true, no reordering is necessary. But if all four tests fail then we have to interchange surface S and S’ in sorted list.

Advantages:

- Simple Implementation: The algorithm is straightforward to understand and implement compared to more complex visibility algorithms.
- Low Memory Usage: Unlike Z-buffering, which requires a substantial amount of memory to store the depth value for every pixel, depth sorting requires much less memory.

limitation:

- Performance Issues with Overlap: If there is a high amount of depth overlap, the system over-renders, leading to performance degradation.

Illumination and shading models:

Shading:

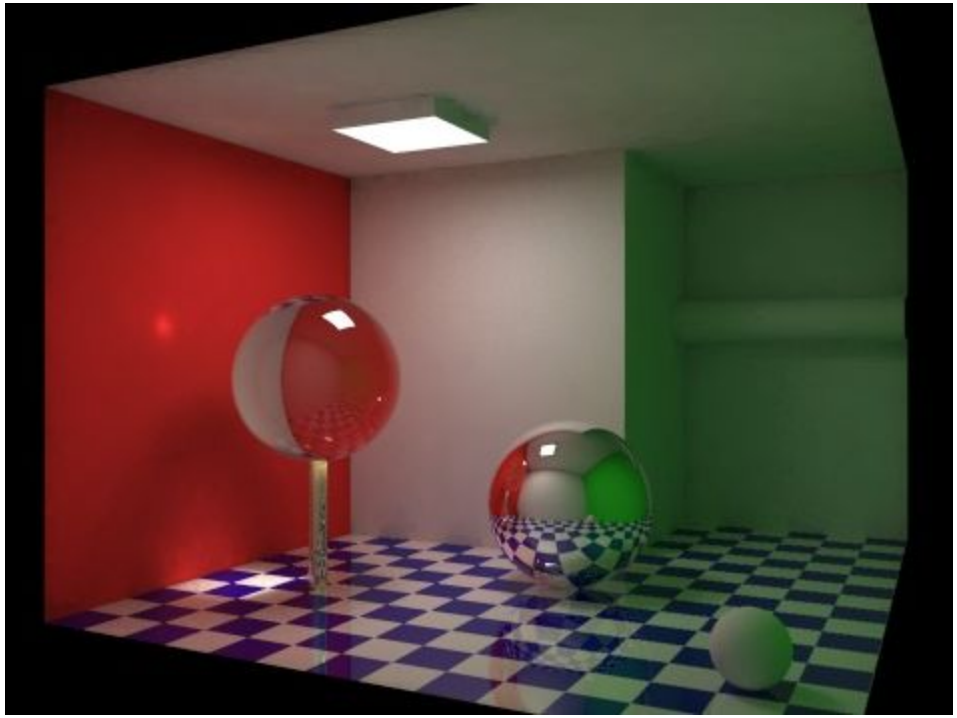
- Darkness where light,particularly sunlight is blocked.
- A variety of color,in particular one,obtained by adding black.
- Shading implementation of the illumination model at the pixel points or polygon surfaces of the graphics object.
- Shading model is used to compute the intensities and colors to display the surface.

Intensity:

- Degree of strength (how much powerful rays of light coming from source to incident on particular surface).
- Quantity of illumination.

Illumination model:

- Also called a lighting/shading model.
- It is used to calculate the intensity of light that we should see at a given point on the surface of an object.



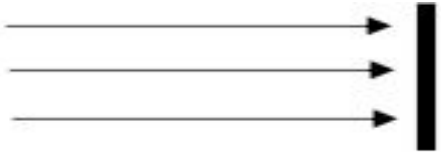
Surface rendering:

- It is a procedure for applying a lighting model to obtain pixel intensities for all projected surface positions in a scene.
- A surface rendering algorithm uses the intensity calculations from an illumination model to determine the light intensity for all projected pixel positions for the various surfaces in a scene.
- Surface rendering can be performed by applying the illumination model to every visible surface point.

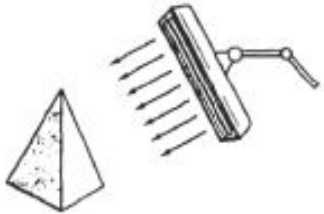
Light sources:

- When we see any object we see reflected light from that object.
- Total reflected light is the sum of contribution from all sources and reflected light from other object that falls on the object.
- So that the surface which is not directly exposed to light may also visible if nearby object is illuminated.
- The simplest model for light source is **point source.**
- Rays from the source then follows radial diverging paths from the source position.

Parallel source: Light rays are all parallel. May be modeled as a point source at infinity (the sun).



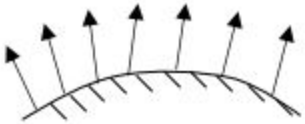
Distributed source: All light rays originate at a finite area in space. A nearby sources such as fluorescent light.



Types of reflection

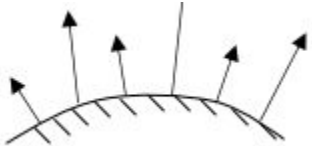
i. Diffuse Reflections:

- Surfaces appear equally bright from all viewing angles since they reflect light with equal intensity in all directions. E.g. rough surfaces



ii. Specular reflection:

- Light reflected with unequal intensity. E.g. shiny surface.



Basic Illumination Models

- Illumination models are used to calculate light intensities that we should see at a given point on the surface of an object.
- Lighting calculations are based on the

Optical properties of surfaces.

The background lighting conditions and

The light source specifications.

Different illumination models are:

1. Ambient Light
2. Diffuse Reflection or Gouraud shading
3. Specular Reflection

Ambient light

- This is simple illumination model.
- Surface getting light from various reflected source i.e. light not coming directly from a light source but coming after getting reflected from other surface.
- Ambient light has no directional properties.
- The amount of ambient light incident on all surfaces and object are constant in all direction.
- The reflected intensity I due to ambient light of any point on the surface is:

$$I = I_a K_a$$

Where, I_a= intensity of ambient light.

K_a= ambient reflection coefficient, $0 \leq K_a \leq 1$.

Diffuse reflection

- When some intensity of light falls on object surface and that surface reflect light in all direction in equal amount then the resulting reflection is diffuse reflection.
- Light reflected with equal intensity in all direction.
- Amount of light hitting the surface depends on the angle between the normal vector and the incident vector of the incoming light.
- Diffuse reflections are constant over each surface independent of viewing directions.
-

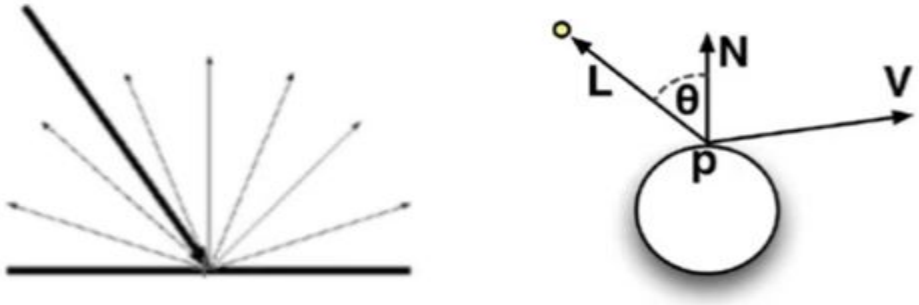


Figure: diffuse Reflection

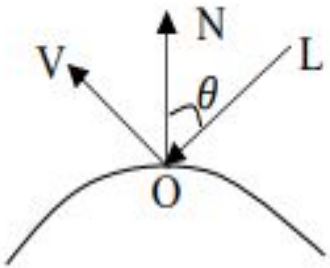
- In diffuse reflection, we consider the surfaces(surface called “ideal diffuse reflectors/Lambertian reflectors),which reflect light with equal intensity in all directions(i.e independent of the viewing direction).The amount of light energy that falls on a given area and hence the intensity of the reflection is proportional to the cosine of the angle of incident(i.e $\cos\theta$.). This kind of surfaces are normally dull. This is known as Lambert’s cosine law.

$$I_{diff} = K_d I_l \cos\theta$$

Where I_l is the intensity of the point light source.

If N is unit vector normal to the surface and L is unit vector in the direction to the point then

Contd….



L=light source

N=normal to surface

V=viewer direction

θ=angle bet^n S & N

The reflected intensity I of any point on surface is

$$I = I_l K_d \vec{N} . \vec{L} = I_l K_d cos\theta$$

Where, K_d= coefficient of diffuse reflection

Net diffuse reflection= Diffuse reflection due to ambient light +Diffuse reflection due to light source = $I_{amb,\ diff} + I_{l,\ diff} = I_a K_a + I_l K_d cos\theta$

$$I_{diff} = K_a I_a + K_d I_l (N.L)$$

Specular reflection and phong model

- In shiny surface we see highlight or bright spot from certain viewing directions called specular reflection. (Light reflected with unequal intensity).
- Mirror like reflection of waves such as light from surface.
- In this process , each incident ray is reflected at the same angle to the normal as the reflected ray , but on the opposite side of the surface normal in the plane formed by incident and reflected rays.
- While looking at illuminated shiny surface , such as polished metal, we see a highlight , or bright spot , at certain viewing directions.
- This phenomenon is called as specular reflection , is the result of total or near total reflection of the incident light in a concentrated region around the specular reflection angle.
- The specular reflection angle equals the angle of incident light.

contd….

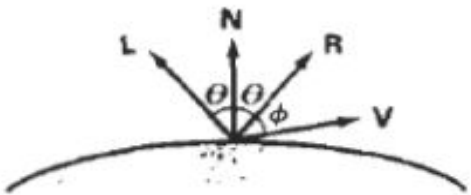
Where,

R is unit vector in direction of reflection(direction of ideal specular reflection)

L is unit vector point towards light source.

N is unit normal vector.

V is the unit vector in viewing direction.



For ideal reflector R and V coincide.

- An empirical model for calculating specular reflection developed by phong , hence called the phong model,sets the intensity of specular reflection proportional to

$\cos^{ns}\emptyset$, where ns -> specular reflection exponent

- For shiny surface ns is large
- For dull surface ns is small

contd..

- For perfect reflector ns is infinite.
- Hence phong specular model reflection

$$I_{l,spec} = K_s I_l (\vec{V}.\vec{R})^{n_s}$$

Where, K_s= specular reflection coefficient
n_s= specular reflection parameter

$$\boxed{I_{l,spec} = K_s I_l cos^{n_s}\phi}$$

Here , $R = 2(N \cdot L)N - L$

Combined diffuse and specular reflection

$$I = I_{diff} + I_{spec}$$

$$I = I_a K_a + I_l K_d (\vec{N}.\vec{L}) + K_s I_l (\vec{V}.\vec{R})^{n_s}$$

contd…

For multiple source we can extend this equation as,

$$I = K_a I_a + \sum_{i=1}^{n} I_l[K_d(N \cdot L) + K_s(V \cdot R)^{ns}]$$

Intensity Attenuation:

- Intensity is attenuated by the factor $1/d$ 2 , where d is the distance that the light has traveled.
- This means that a surface close to the light source (small d) receives a higher incident intensity from the source than a distant surface (large d).
- Graphics package have a general inverse quadratic attenuation function

$$f(d) = \min(1, \frac{1}{a_0 + a_1 d + a_2 d^2})$$

contd..

User can fiddle with the coefficients a_0, a_1, a_2 to obtain a variety of lighting effects for a scene. The a_0 can be adjusted to prevent f(d) from becoming too large when d is very small. This is an effective method for limiting intensity values when a single light source is used to illuminate a scene.

With a given set of attenuation coefficients, we can limit the magnitude of the attenuation function to 1 with the calculation

$$f(d) = \frac{1}{a_0 + a_1 d + a_2 d^2}$$

Using this function, we can then write our basic illumination model as

$$I = K_a I_a + \sum_{i=1}^{n} f(d_i) I_{Li} [k_d (N.L_i) + k_s (N.H_i)^{n_s}]$$

Where d_i is the distance light has traveled from light source *i*.

Polygon Surface Rendering Methods:

- **Polygon rendering** is the process of converting 3D polygon surfaces into a **2D image** by calculating the **color and intensity of pixels** on the polygon. The aim is to make 3D objects appear realistic by simulating **lighting, shading, and surface reflection**.
- Shading model is used to calculate the intensities and color to display the surface.
- It involves applying **shading, lighting, and textures** to create a realistic or stylized appearance.
- Methods of polygon renderings:

    1) constant shading (flat shading)

    2) gouraud shading

    3) phong shading

    4) fast phong shading

1. Constant shading

- A fast and simple method for rendering an object with polygon surfaces is constant intensity shading.
- In this method, a single intensity is calculated for each polygon and the intensity is applied to all the points of surface of polygon. Hence, all the points over the surface of the polygon are displayed with same intensity value.
- Constant shading is useful for quickly displaying the general appearance of a curved surfaces.
- This approach is valid if:

i. The object is a polyhedron and is not an approximation of an object with a curved surface.

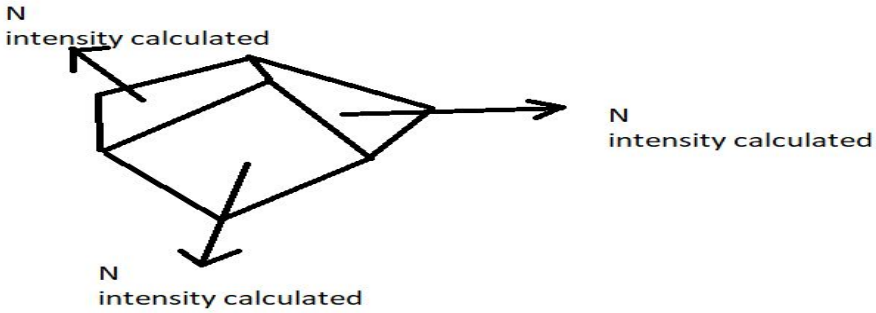
ii All light source illuminating the object are sufficiently far from the surface.

iii The viewing position is constant over the surface

contd..

Advantage:

- Constant shading can be useful for quickly displaying the general appearance of a curved surface.



Disadvantage:

- **Sharp intensity discontinuation** is seen in the border between two polygon.

Gouraud shading(intensity interpolation method)

- Developed by Henri Gouraud.
- It renders the polygon surface by linearly interpolating intensity values across the surface.
- Intensity values for each polygon are matched with the values of adjacent polygon along the common edge, thus eliminating the intensity discontinuities that occur in flat shading.

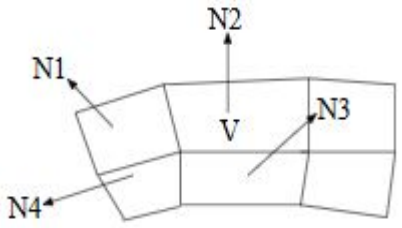
Steps:

i) determine the average unit normal vector at each vertex of the polygon.

ii) apply an illumination model at each vertex to obtain the light intensity at that vertex.

iii) interpolate the vertex intensities over the projected area of the polygon.

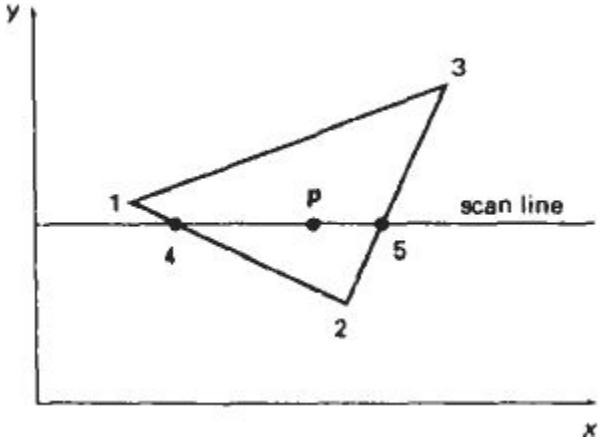
contd…



$$\vec{N}v = \frac{\sum_{k=1}^{n} \vec{N}k}{\left|\sum_{k=1}^{n} \vec{N}k\right|}$$

- At each polygon vertex , the normal vector is obtained by averaging the normal vector of all polygons in the surface that share that vertex.
- Once normal vector is obtained at the vertex intensity at that vertex is calculated using illumination model.
- After all the vertex intensity have been computed , linear interpolation is used to compute intensity along the polygon surface using scan line.

contd..



- For each scan line the intensity at the intersection of scan line with a polygon edge is obtained by linear interpolation of intensity at the endpoint of that edge

$$I_4 = \frac{y_4 - y_2}{y_1 - y_2} I_1 + \frac{y_1 - y_4}{y_1 - y_2} I_2$$

- Similarly intensity at position 5 i.e I5 can be calculated using intensity position 2 and 3.

contd..

$$I_5 = \frac{y_5 - y_2}{y_3 - y_2} I_3 + \frac{y_3 - y_5}{y_3 - y_2} I_2$$

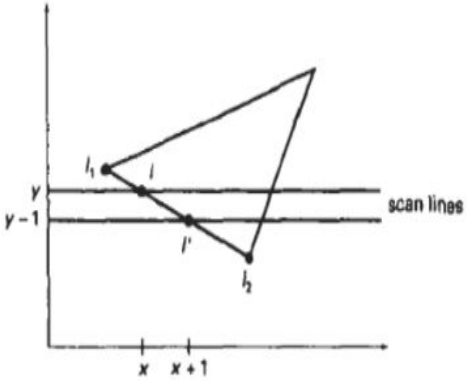
- For interior point p interpolated from bounding intensities at point 4 and 5

$$I_p = \frac{x_5 - x_p}{x_5 - x_4} I_4 + \frac{x_p - x_4}{x_5 - x_4} I_5$$

- For next scan line

for next scan line y - 1

$$I' = I + \frac{I_2 - I_1}{y_1 - y_2}$$



contd..

Hence , each successive intensity value down the edge is computed simply by adding the constant term $\frac{I_2 - I_1}{y_1 - y_2}$ to the previous intensity value.

- Similarly incremental calculation can be used to obtain intensity at successive horizontal position along each scan line.

Advantages:

- Removes intensity discontinuities at the edges as compared to constant shading.

Disadvantages:

- Causes bright or dark intensity streaks called match bands to appear on the surface.

Mach bands

- **Mach bands** are a *visual illusion* where the human eye sees **extra bright and dark (bands/stripes)** near the boundary between two different shades, even if the change is actually smooth.
- Mach bands are an optical illusion consisting of an image of two wide bands,one light and one dark,separated by a narrow strip with a light-to-dark gradient.The human eye perceives two narrow bands of different brightness either side of the gradient that are not present in the original image.

Why do Mach bands happen?

Our eyes and brain **enhance contrast** at edges to help us detect boundaries clearly.

So when there is a gradual change in brightness:

- the eye makes the region near the boundary look **brighter**
- and the region just next to it look **darker**

  This effect is caused by **human perception**, not by real lighting.

contd…

Example: If a surface changes from dark → light smoothly:

Your eye may see: a **bright strip** near the light side
a **dark strip** near the dark side

- Even though those strips are not really there.

- Mach bands are noticeable in shading methods like **Gouraud shading**, because: intensity is interpolated across polygons , slight intensity differences at polygon boundaries appear like visible bands

Figure: Mach Bands Effect

Phong shading model

- Developed by Phong Bui Tuong.

- also known as normal vector interpolation rendering , that interpolates normal vector instead of intensity values which results more accurate calculation of intensity values hence a more realistic display of surface highlights and great reduction in match band effect.

- It renders the polygon surface by linearly interpolating normal vector across the surface.

Steps:

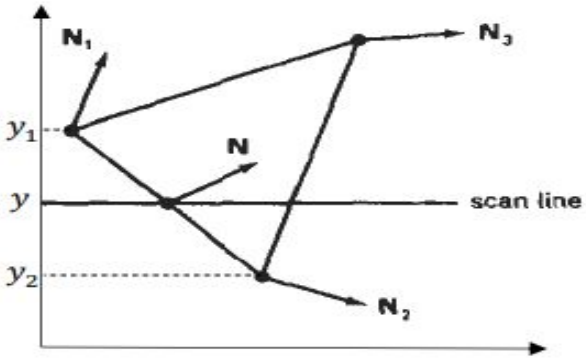
1. Determine the average unit normal vector at each vertex of polygon.

$$\vec{N}_{avg} = \frac{\sum_{i=1}^{n} \vec{N}_i}{\left|\sum_{i=1}^{n} \vec{N}_i\right|}$$

contd..

2. Linearly interpolate the vertex normal over the projected area of the polygon.

3. Apply illumination model at position along each scan line to calculate pixel intensities using interpolated normal vectors.



$$\vec{N} = \frac{y - y_2}{y_1 - y_2}\vec{N}_1 + \frac{y_1 - y}{y_1 - y_1}\vec{N}_2$$

contd..

Advantages:

- It displays more realistic highlights on a surface.

- It greatly reduces the Mach band effect.

- It gives more accurate results.

Disadvantages:

- It requires more calculation and greatly increases the cost of shading steeply.

- Computationally expensive.

Fast phong shading

- Fast phong shading approximates the intensity calculations using a Taylor-series expansion and triangular surface patches.
- Surface normal at any point (x, y) over a triangle as; $N = Ax + By + C$.

A, B, C are determined from the three vertex equations:

$$N_k = Ax_k + By_k + C, \qquad k = 1,2,3$$

With (x_k, y_k) denoting a vertex position.
Omitting the reflectivity and attenuation parameters,

$$I_{diff}(x,y) = \frac{L \cdot N}{|L||N|} = \frac{L \cdot (Ax + By + C)}{|L||Ax + By + C|} = \frac{(L \cdot A)x + (L \cdot B)y + L \cdot C}{|L||Ax + By + C|}$$

contd…

We can write this,

$$I_{diff}(x,y) = \frac{ax + by + c}{(dx^2 + exy + fy^2 + gx + hy + i)^{1/2}} \ldots\ldots\ldots..(i)$$

Where, a, b, c, d are used to represent the various dot product. For e.g.

$$a = \frac{L \cdot A}{|L|}$$

Finally, denominator in eq. (i) can be expressed as a Taylor-series expansion and retain terms up to second degree in x & y. This yields

$$I_{diff}(x,y) = T_5x^2 + T_4xy + T_3y^2 + T_2x + T_1y + T_0$$

Where, each T_k is a function of parameters a, b, c, and so forth.

Comparison of polygon rendering method

Flat shading	Gouraud shading	Phong shading
Computes illumination once per polygon and apply it to whole polygon.	Computes illumination at vertices and interpolates.	Applies illumination at every point of polygon surface.
Creates discontinuous in color.	Interpolates colors along edges and scan line.	Interpolates normal instead of colors.
Problems of Mach bands.	Handles Mach bands problem found in flat shading.	Removes Mach bands completely.
Low cost.	Not so expensive.	More expensive than gouraud shading.
Requires very less processing and is fast in time.	Required moderate processing time.	Requires complex processing and is slower but is more efficient as compared to other shading methods.
Lighting equation used once per polygon.	Lighting equation used at each vertex.	Lighting equation used at each pixel.

Color models

- Color model is an orderly system for creating a whole range of color from a small set of primary colors.
- There are two types of color models , those that are subtractive and those that are additive.
- Additive colors model use light to display color and subtractive model use Printing inks.
- Colors perceived in additive model are the result of transmitted light and colors perceived in subtractive model are the result of reflected light.
- There are several established colors model used in computer graphics.
- The two most common models are the RGB model for computer display (red,green and blue) and CMY(cyan,magenta ,yellow) for printing.

contd..

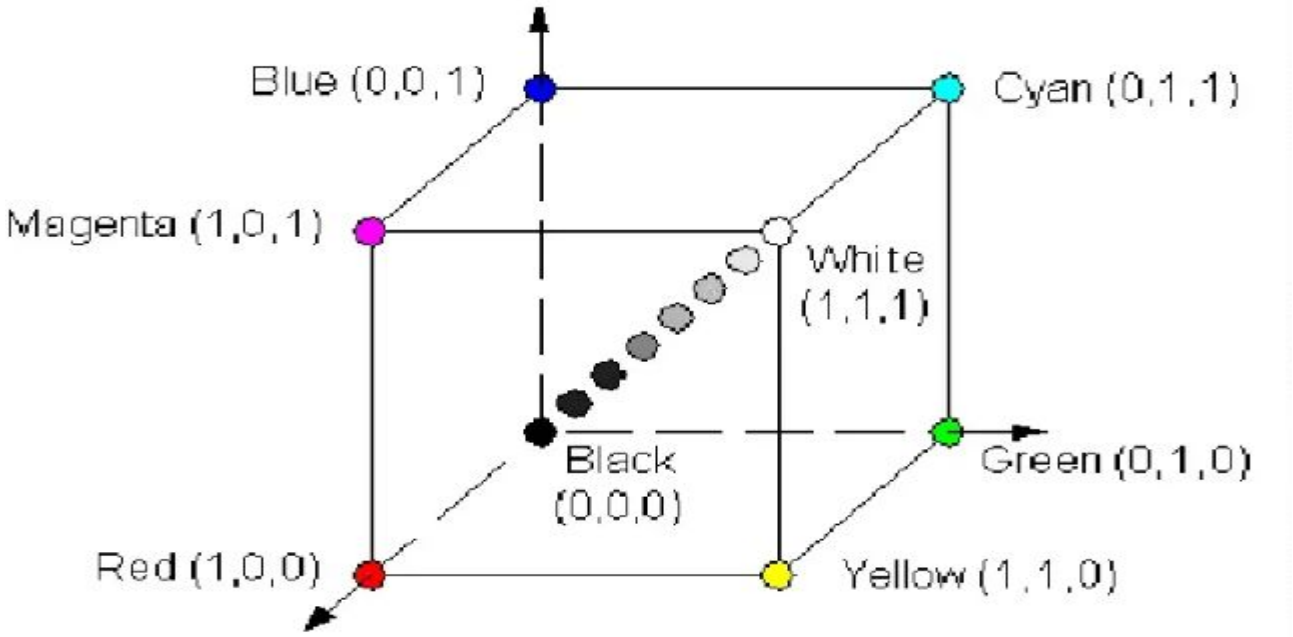
RGB color model:

- The RGB color model is one of the most widely used color representation method in computer graphics.
- It use a color coordinate system with three primary colors:

```
R(red), G(green), B(blue)
```

- Each primary color can take an intensity value ranging from 0(lowest) to 1(highest).
- Mixing these three primary colors at different intensity levels produces a variety of colors.
- The collection of all the colors obtained by such a linear combination of red, green and blue forms the cube shaped RGB color space.

contd…


Blue (0,0,1)
Cyan (0,1,1)
Magenta (1,0,1)
White
(1,1,1)
Black
(0,0,0)
Green (0,1,0)
Red (1,0,0)
Yellow (1,1,0)

contd..

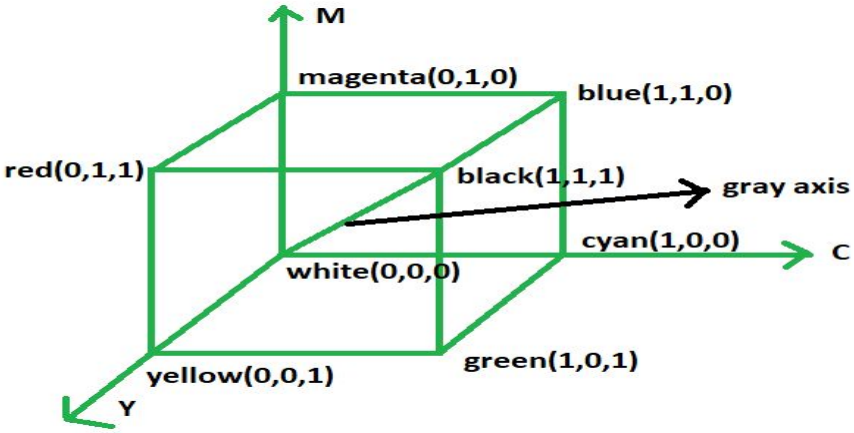
- The corner of RGB color cube that is at the origin of the coordinate system corresponds to black, whereas the corner of the cube that is diagonally opposite to the origin represents white.
- The diagonal line connecting black and white corresponds to all the gray colors between black and white, which is also known as **gray axis**.
- In the RGB color model, an arbitrary color within the cubic color space can be specified by its color coordinates: (r, g.b).
- Color specification using the RGB model is an **additive process**. We begin with black and add on the appropriate primary components to yield a desired color.
- The concept RGB color model is used in **Display monitor**.

CMY color:

- The CMY color model use a **subtraction process** and this concept is used in the **printer**.
- In CMY model, we begin with white and take away the appropriate primary components to yield a desired color.
- **Example:** If we subtract red from white, what remains consists of green and blue which is cyan. The coordinate system of CMY model use the three primaries' complementary colors:

  `C(cyan), M(magenta) and Y(yellow)`

-



contd…

- The corner of the CMY color cube that is at (0, 0, 0) corresponds to white, whereas the corner of the cube that is at (1, 1, 1) represents black.
- The following formulas summarize the conversion between the two color models:
-

$$\begin{bmatrix} R \\ G \\ B \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} - \begin{bmatrix} C \\ M \\ Y \end{bmatrix} \qquad \begin{bmatrix} C \\ M \\ Y \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} - \begin{bmatrix} R \\ G \\ B \end{bmatrix}$$

CMYK color model

- **CMYK** stands for **Cyan, Magenta, Yellow, and Key (Black)** — the four inks used in full-color printing.
- CMYK is a **subtractive color model** used for **printing on physical surfaces** like paper and fabrics. In subtractive color mixing, inks absorb (subtract) light from white paper to form colors.
- Colors are produced by **layering tiny dots** of cyan, magenta, yellow, and black ink. Different combinations of these inks create a wide range of printable colors.

- **Black** (K) is used because combining cyan, magenta, and yellow alone doesn't produce a true deep black and is less efficient in ink usage
- The black plate (K) adds **depth, contrast, and detail** to prints.

- It also **reduces ink consumption** and prevents slowing down the print process by avoiding excessive mixing of the three colored inks
-

Thank you